Teste de Software

Trabalho Prático: Detecção de Bad Smells e Refatoração Segura

Aluno: Pedro Augusto Santos Seabra

Matrícula: 812784

Repositório: https://github.com/pedroseabra27/bad-smells-js-refactoring

1. Análise de Smells

O arquivo analisado foi `src/ReportGenerator.js`, cujo único método, `generateReport`, faz tudo: monta cabeçalho, filtra itens por perfil, marca prioridade, formata linhas, soma o total e monta o rodapé. Encontrei três Bad Smells principais.

1.1 Long Method (Método Longo)

`generateReport` tem cerca de 57 linhas e pelo menos 5 responsabilidades diferentes (cabeçalho, filtro por permissão, regra de prioridade, formatação do corpo, total/rodapé).

- **Manutenção:** para mudar qualquer detalhe (ex.: o rodapé do HTML) é preciso entender o método inteiro. O risco de quebrar outra parte sem querer é alto.
- **Testes:** não dá para testar uma regra isolada (ex.: "usuário comum só vê itens ≤ 500") sem gerar um relatório completo e procurar substrings na saída.

1.2 Condicionais Complexas / Switch Statements repetidos

A pergunta `reportType === 'CSV' / 'HTML'` aparece **4 vezes** (cabeçalho, corpo do ADMIN, corpo do USER e rodapé), e dentro do laço ela está aninhada com `user.role === 'ADMIN' / 'USER'` e com `item.value > 1000` / `item.value <= 500`. São até 4 níveis de `if` aninhados.

- **Manutenção:** adicionar um formato novo (ex.: JSON) obriga a editar 4 lugares diferentes. Esquecer um deles gera um relatório quebrado. Isso fere o Princípio Aberto/Fechado.
- **Testes:** cada combinação de formato × perfil × faixa de valor é um caminho diferente, então o número de casos de teste cresce de forma multiplicativa.

1.3 Código Duplicado + Números Mágicos

- A linha que monta o CSV (`` `${item.id},${item.name},${item.value},${user.name}\n` ``) está copiada nos ramos do ADMIN e do USER, e `total += item.value` aparece 4 vezes.
- Os valores `1000` (limite de prioridade) e `500` (limite do usuário comum), além das strings `'ADMIN'`, `'USER'`, `'CSV'`, `'HTML'`, aparecem soltos no código, sem nome.
- **Manutenção:** se o formato da linha CSV mudar, é preciso lembrar de alterar as duas cópias. Se o limite de 500 mudar, é preciso adivinhar o que aquele número significa e procurá-lo pelo código.
- **Testes:** a duplicação abre espaço para bugs que só aparecem em um dos ramos. Inclusive, a suíte de testes revelou um bug assim no original: no HTML do ADMIN, a linha sem prioridade é gerada como `<tr >` (com espaço sobrando), enquanto no ramo do USER sai `<tr>`. O teste *"deve gerar um relatório HTML completo para Admin"* falha no código original por causa disso.

2. Relatório da Ferramenta (ESLint + eslint-plugin-sonarjs)

Configuração usada (`.eslintrc.json`), com as regras recomendadas do SonarJS:

```
{
  "extends": ["eslint:recommended", "plugin:sonarjs/recommended"],
  "plugins": ["sonarjs"],
  "rules": {
    "sonarjs/cognitive-complexity": ["error", 5],
    "sonarjs/no-identical-functions": "error",
    "sonarjs/no-duplicated-branches": "error"
  }
}
```

Resultado de `npx eslint src/ReportGenerator.js` **antes** da refatoração:

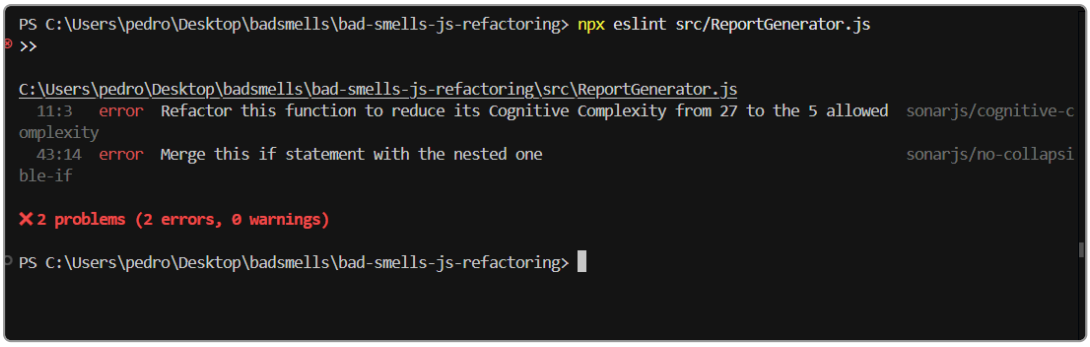


Figura 1: ESLint no código original (2 erros)

Como o SonarJS ajudou:

- **Complexidade cognitiva (27):** na leitura manual eu percebi que o método era "complicado", mas a ferramenta transformou isso em um número objetivo. Ela soma pontos para cada `if` / `else if` / `for` e soma mais ainda quando eles estão aninhados. 27 é mais de 5 vezes o limite configurado e quase o dobro do limite padrão de mercado (15). Isso deixa claro que o problema não é só o tamanho do método, e sim o **aninhamento**.
- **`no-collapsible-if` (linha 43):** a ferramenta apontou o `else if (user.role === 'USER')` seguido de `if (item.value <= 500)`, dois `if` que poderiam ser um só. É um detalhe fácil de passar despercebido na leitura manual, porque o código "parece certo".
- Por outro lado, a ferramenta **não** apontou os números mágicos nem a duplicação das linhas de CSV (são trechos curtos, abaixo do limite das regras de duplicação). Ou seja, ferramenta e análise manual se complementam.

3. Processo de Refatoração

O smell mais crítico era a **condicional complexa e repetida** (formato × perfil), que é a principal causa da complexidade cognitiva 27.

Antes (`src/ReportGenerator.js`)

```
generateReport(reportType, user, items) {
  let report = '';
  let total = 0;
  if (reportType === 'CSV') { /* cabeçalho CSV */ }
  else if (reportType === 'HTML') { /* cabeçalho HTML */ }

  for (const item of items) {
    if (user.role === 'ADMIN') {
      if (item.value > 1000) { item.priority = true; }
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        const style = item.priority ? 'style="font-weight:bold;"' : '';
        report += `<tr ${style}><td>${item.id}</td>...</tr>\n`;
        total += item.value;
      }
    } else if (user.role === 'USER') {
      if (item.value <= 500) {
        if (reportType === 'CSV') {
          report += `${item.id},${item.name},${item.value},${user.name}\n`;
          total += item.value;
        } else if (reportType === 'HTML') {
          report += `<tr><td>${item.id}</td>...</tr>\n`;
          total += item.value;
        }
      }
    }
  }
  if (reportType === 'CSV') { /* rodapé CSV */ }
  else if (reportType === 'HTML') { /* rodapé HTML */ }
  return report.trim();
}
```

Depois (`src/ReportGenerator.refactored.js`)

```
const USER_MAX_ITEM_VALUE = 500;
const PRIORITY_THRESHOLD = 1000;

class CsvFormatter {
  header() { return 'ID,NOME,VALOR,USUARIO\n'; }
  row(item, user) { return `${item.id},${item.name},${item.value},${user.name}\n`; }
  footer(total) { return `\nTotal,,\n${total},,\n`; }
}

class HtmlFormatter {
  header(user) { /* <html>... <h2>Usuário: ...</h2> <table> */ }
  row(item, user) { /* <tr> com negrito se ADMIN e prioritário */ }
  footer(total) { /* </table><h3>Total: ...</h3></body></html> */ }
}

const FORMATTERS = { CSV: new CsvFormatter(), HTML: new HtmlFormatter() };

const VISIBILITY_RULES = {
  ADMIN: () => true,
  USER: (item) => item.value <= USER_MAX_ITEM_VALUE,
};

generateReport(reportType, user, items) {
  const formatter = this.getFormatter(reportType);
  const visibleItems = this.filterVisibleItems(user, items);

  if (user.role === ROLES.ADMIN) {
    this.markPriorityItems(visibleItems);
  }

  const body = visibleItems.map((item) => formatter.row(item, user)).join('');
  const total = this.calculateTotal(visibleItems);

  return (formatter.header(user) + body + formatter.footer(total)).trim();
}
```

Técnicas aplicadas

Técnica (Fowler)	Onde / Para quê
Replace Conditional with Polymorphism	Criei `CsvFormatter` e `HtmlFormatter` com a mesma interface (`header` , `row` , `footer`). Os 4 `if (reportType === ...)` sumiram. Um formato novo vira só uma classe nova.
Decompose Conditional	A regra "quem vê o quê" virou o mapa `VISIBILITY_RULES` , usado em `filterVisibleItems` . Isso eliminou o `if` de perfil aninhado dentro do laço (e o `no-collapsible-if`).
Extract Method	`getFormatter` , `filterVisibleItems` , `markPriorityItems` e `calculateTotal` . Cada um tem uma responsabilidade e um nome que explica a intenção.
Replace Magic Number with Symbolic Constant	`500` → `USER_MAX_ITEM_VALUE` , `1000` → `PRIORITY_THRESHOLD` , `'ADMIN'/'USER'` → `ROLES` .
Remove Duplicated Code	A linha CSV e o `total += item.value` (repetido 4 vezes) passaram a existir em um único lugar (`CsvFormatter.row` e `calculateTotal`).
Introduce Null Object	`EmptyFormatter` mantém o comportamento original para tipos de relatório desconhecidos (saída vazia) sem precisar de `if` .

Resultado

Métrica	Antes	Depois
Complexidade cognitiva de `generateReport`	27	≤ 1
Erros do ESLint/SonarJS	2	**0**
Repetições de `if (reportType === ...)`	4	0
Testes da suíte (arquivo refatorado)	—	**5/5 passando**

Resultado de `npx eslint src/ReportGenerator.refactored.js` : **nenhum erro** (o ESLint não imprime nada; a mensagem "ESLint: 0 erros" vem do `echo` executado em seguida).

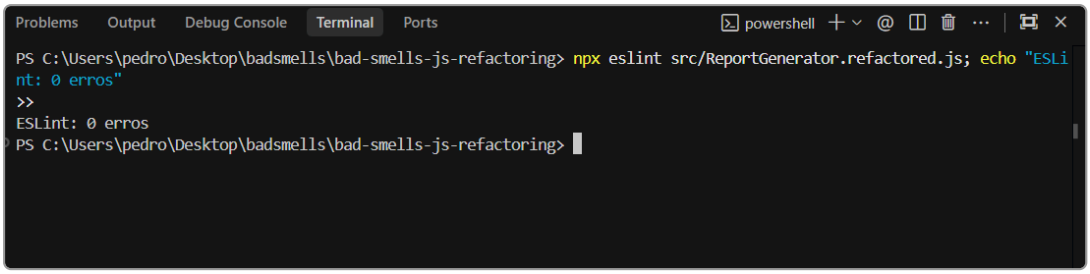


Figura 2: ESLint no código refatorado (0 erros)

Resultado de `npm test` para o arquivo refatorado (rede de segurança):

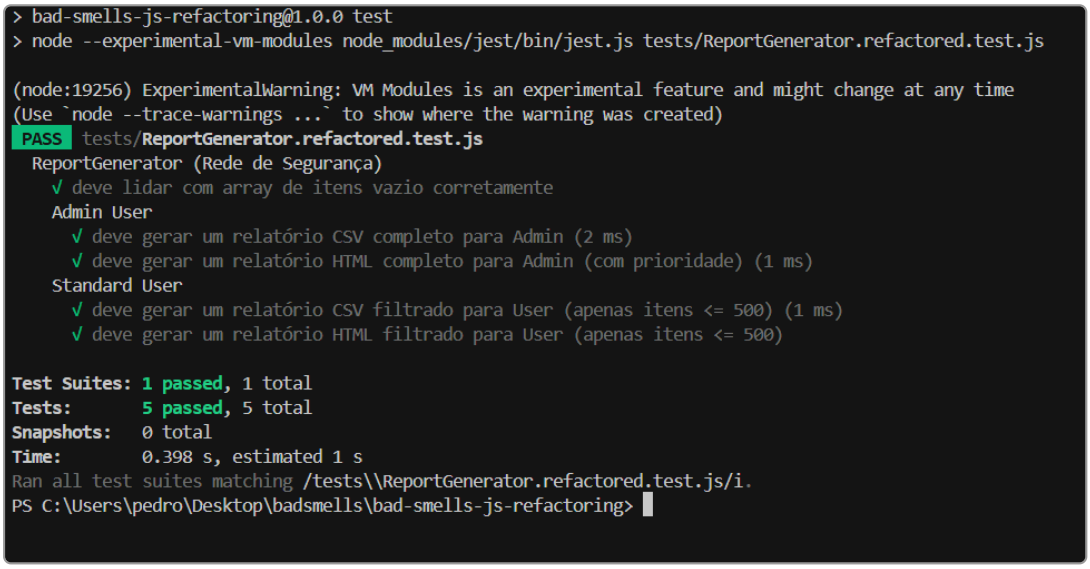


Figura 3: Testes do código refatorado (5/5 passando)

4. Conclusão

A suíte de testes foi a rede de segurança de todo o processo. Como ela verifica o **comportamento** (a saída do relatório) e não a implementação, consegui reescrever o método inteiro, trocando `if`s por classes, laços por `filter` / `map` / `reduce` e criando novos métodos, sem alterar uma linha dos testes. Rodei `npm test` a cada passo e confirmei que nada tinha quebrado. Sem esses testes, uma mudança desse tamanho seria um "chute". Com eles, ela virou uma sequência de passos pequenos e verificáveis. A suíte inclusive revelou um bug no código original (o `<tr>` com espaço no HTML do administrador), que a versão refatorada não tem.

Reduzir os Bad Smells melhorou a qualidade do software de forma mensurável: a complexidade cognitiva caiu de 27 para praticamente 1 e o linter passou a não apontar nenhum erro. Na prática, o código ficou mais **legível** (cada método diz o que faz), mais **extensível** (um novo formato é só uma classe nova, sem mexer no gerador) e mais **testável** (regras como visibilidade e prioridade podem ser testadas isoladamente). Ferramentas como o `eslint-plugin-sonarjs` tornam esses problemas visíveis e objetivos, mas é a combinação de análise manual, análise automática e testes que permite refatorar com confiança.

Referências

- FOWLER, Martin. *Refatoração: Aperfeiçoando o Design de Códigos Existentes*. 2. ed. Novatec, 2019.
- Catálogo de Refatoração: https://refactoring.com/catalog/
- eslint-plugin-sonarjs: https://github.com/SonarSource/eslint-plugin-sonarjs